

Sistemas Operacionais

Prof. Rafael Obelheiro
rafael.obelheiro@udesc.br



Gerência de Entrada e Saída

Introdução

- O controle dos dispositivos de E/S é uma das principais funções do SO
- O SO deve oferecer ao usuário uma interface simples e fácil de usar, ocultando na medida do possível as particularidades de dispositivos específicos
- O gerenciamento de E/S responde por uma fração significativa do código em um SO
 - ▶ variedade de dispositivos que necessitam de suporte

```
### exemplo: conta drivers disponíveis como módulos de kernel
$ find /lib/modules/'uname -r'/kernel/drivers -name '*.ko' | wc -l
4307
```

Sumário

- 1 Introdução
- 2 Princípios de hardware de E/S
- 3 Princípios do software de E/S
- 4 Camadas do software de E/S
- 5 Discos
- 6 Discos de estado sólido (SSDs)
- 7 E/S no Linux

Sumário

- 1 Introdução
- 2 Princípios de hardware de E/S
- 3 Princípios do software de E/S
- 4 Camadas do software de E/S
- 5 Discos
- 6 Discos de estado sólido (SSDs)
- 7 E/S no Linux

Dispositivos de E/S

- *Lembrete: o ponto de vista é o da programação do dispositivo, não da sua construção física*
- Existem duas grandes classes de dispositivos
- **Orientados a blocos:** transferem dados em unidades de tamanho fixo (blocos)
 - blocos variam de 512 bytes a 32 KB (tipicamente)
 - endereçamento também é por bloco (ler o bloco n)
- **Orientados a caracteres:** transferem dados em sequências (fluxos) de caracteres
 - não podem ser endereçados
- Nem todos os dispositivos se encaixam nessas classes
 - um relógio só gera interrupções a intervalos periódicos

Velocidade dos dispositivos

- As velocidades dos dispositivos variam bastante

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Scanner at 300 dpi	1 MB/sec
Digital camcorder	3.5 MB/sec
4x Blu-ray disc	18 MB/sec
802.11n Wireless	37.5 MB/sec
USB 2.0	60 MB/sec
FireWire 800	100 MB/sec
Gigabit Ethernet	125 MB/sec
SATA 3 disk drive	600 MB/sec
USB 3.0	625 MB/sec

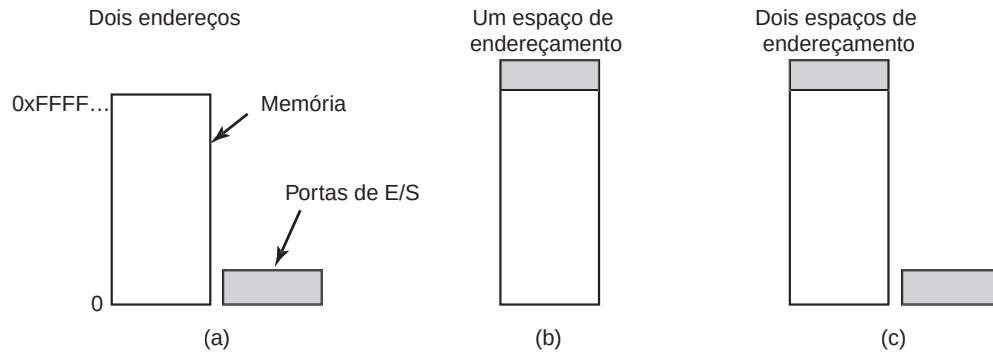
Controladores de dispositivos

- Dispositivos tipicamente têm dois grandes componentes
 - eletrônico: programação
 - mecânico: dispositivo propriamente dito
- O componente eletrônico é o **controlador**
 - o mesmo controlador pode tratar vários dispositivos idênticos
- Tarefas do controlador
 - leitura e escrita serial de bits
 - conversão entre fluxos de bits e blocos fixos
 - verificação e correção de erros
 - tornar um bloco ou caracter disponível para ser copiado para a memória

E/S mapeada em memória (1)

- Os controladores de dispositivos precisam ser endereçados de alguma maneira
 - registradores de dados, status e controle
- Existem duas formas básicas de endereçamento de E/S
- **Portas de E/S:** endereços separados para E/S
 - exigem instruções específicas de máquina (IN, OUT)
- **E/S mapeada em memória:** endereços de E/S são os mesmos endereços de memória
 - ocupam faixas distintas no espaço de endereçamento
 - utilizam as mesmas instruções de máquina que referenciam a memória

E/S mapeada em memória (2)



- (a) Espaços de memória e E/S separados
- (b) E/S mapeada em memória
- (c) Endereçamento híbrido (memória para dados, portas para controle/status)

E/S mapeada em memória (3)

Exemplo de E/S com portas: leitura do setor 0x123456 de um disco ATA (modo LBA com endereços de 28 bits)

1. Escreve um byte na porta 0x1f6 para escolher drive e endereçamento LBA
2. Escreve 1 (número de setores) na porta 0x1f2
3. Envia o número do setor a prestações:
 - 3.1 Escreve 0x56 (byte menos significativo) na porta 0x1f3
 - 3.2 Escreve 0x34 na porta 0x1f4
 - 3.3 Escreve 0x12 (byte mais significativo) na porta 0x1f5
4. Escreve 0x20 (comando READ SECTORS) na porta 0x1f7
5. Espera uma interrupção ou lê a porta 0x1f7 em loop até que o controlador sinalize o fim da operação
6. Copia 256 valores de 16 bits da porta 0x1f0 para um buffer
 - ▶ em loop, um valor por vez

E/S mapeada em memória (4)

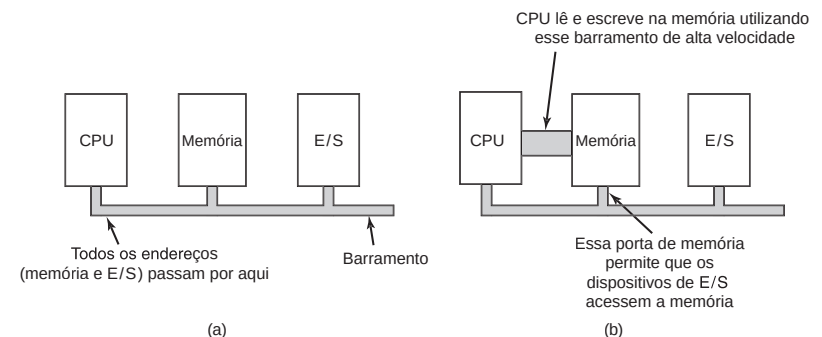
Vantagens de E/S mapeada em memória:

- Registradores do controlador são posições de memória e podem ser referenciados como variáveis em C
 - ▶ dispensa o uso de Assembly para escrever drivers
- Dispensa o uso de mecanismos especiais de proteção para impedir o acesso de processos de usuário aos dispositivos
 - ▶ mecanismos de proteção de memória são suficientes
- Qualquer instrução que referencie memória pode ser usada para manipular os registradores do controlador
 - ▶ torna a programação mais flexível e (possivelmente) eficiente

E/S mapeada em memória (5)

Desvantagens de E/S mapeada em memória:

- Complica o gerenciamento do cache de memória
 - ▶ o cache deve ser desabilitado para endereços referentes aos registradores de dispositivos
- Adiciona complexidade quando existem múltiplos barramentos de memória
 - ▶ todos os endereços têm de ser inspecionados pela memória e pelos controladores de dispositivo



Acesso direto à memória (DMA)

- Controladores de dispositivos trabalham com unidades de blocos ou bytes
 - ▶ o controlador precisa ser continuamente atendido pelo SO para realizar uma transferência de dados
 - ▶ o desperdício de CPU é muito grande
- Uma solução alternativa é o uso de acesso direto à memória (DMA, *direct memory access*)
 - ▶ as transferências de dados são feitas diretamente entre controlador e memória, sem passar pela CPU
 - ▶ requer um controlador específico (controlador de DMA)
 - ▶ o controlador de DMA tem acesso direto ao barramento

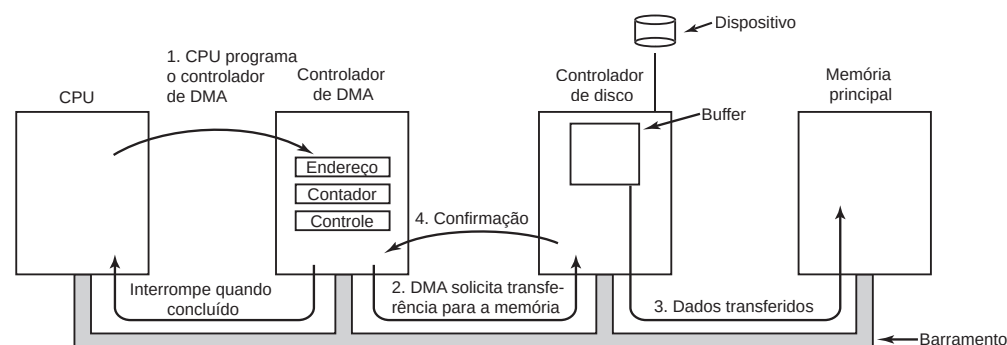
Leitura de dados sem DMA

1. O controlador efetua a leitura do bloco de dados (serialmente, bit a bit)
2. O controlador faz a verificação e correção de erros
3. O controlador causa uma interrupção
4. O SO lê o bloco do buffer do controlador, um byte ou bloco por vez
5. O SO armazena os dados lidos na memória
6. Repete até que a leitura esteja concluída

Leitura de dados com DMA

1. A CPU programa o controlador de DMA, indicando
 - ▶ quais os blocos que devem ser lidos
 - ▶ o endereço inicial na memória
 - ▶ o total de bytes/blocos a transferir
2. O controlador de DMA solicita a leitura do próximo bloco ao controlador de disco
3. O controlador de disco interrompe o controlador de DMA quando tiver lido um bloco
4. O controlador de DMA comanda a transferência dos dados do buffer do controlador de disco para a memória
5. Se houver mais dados a ler, volta ao passo 2; senão, interrompe a CPU sinalizando o final da transferência

Transferência de dados com DMA

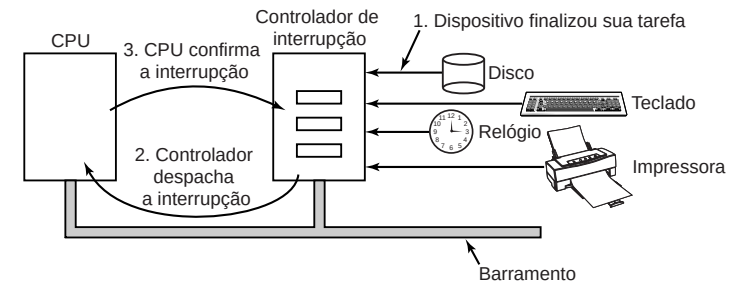


Interrupções revisitadas (1)

Funcionamento de interrupções:

- Quando um dispositivo de E/S finaliza seu trabalho, ele gera uma interrupção
- Isso é feito através do envio de um sinal pela linha de barramento à qual o dispositivo está associado
- O sinal é detectado pelo chip controlador de interrupções localizado na placa-mãe
- O sinal de interrupção faz com que a CPU pare de executar a tarefa atual e vá tratar a interrupção

Interrupções revisitadas (2)

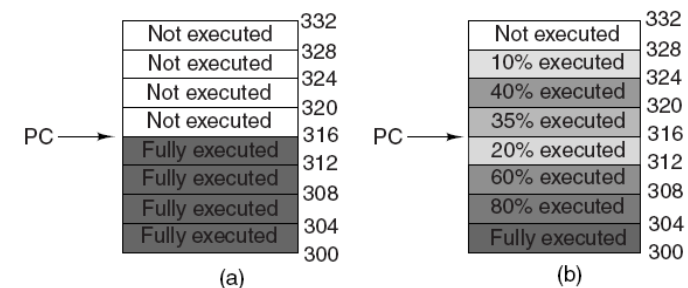


- A confirmação (3) sinaliza ao controlador que a interrupção foi atendida
 - ▶ o controlador de interrupção avisa ao dispositivo e pode liberar outra interrupção pendente

Interrupções precisas e imprecisas (1)

- Em arquiteturas pipelined e superescalares, várias instruções podem estar sendo executadas quando ocorre uma interrupção
 - ▶ qual o estado da CPU quando a interrupção acontece?
- **Interrupções precisas** deixam o sistema em um estado bem definido
 1. O PC é salvo em um lugar conhecido
 2. Todas as instruções antes do PC foram completamente executadas
 3. Nenhuma instrução depois do PC foi executada
 4. O estado de execução da instrução apontada pelo PC é conhecido
- **Interrupções imprecisas** não atendem a esses requisitos
- Interrupções precisas complicam o hardware e simplificam o SO
 - ▶ abordagem adotada pela maioria das arquiteturas atuais

Interrupções precisas e imprecisas (2)



(a) interrupção precisa

(b) interrupção imprecisa

Sumário

- 1 Introdução
- 2 Princípios de hardware de E/S
- 3 Princípios do software de E/S
- 4 Camadas do software de E/S
- 5 Discos
- 6 Discos de estado sólido (SSDs)
- 7 E/S no Linux

Objetivos do software de E/S (2)

- **Transferência síncrona vs assíncrona:** SO precisa implementar chamadas de sistema síncronas (bloqueantes) usando operações de E/S assíncronas
 - operações assíncronas: o chamador não espera pelo resultado, ele solicita a operação e mais tarde recupera o resultado
- **Buffers:** operações de E/S exigem que o SO gerencie buffers de armazenamento temporário de dados
 - transferências de dados entre buffers podem ter um impacto significativo no desempenho

Objetivos do software de E/S (1)

- **Independência do dispositivo:** programas de usuário devem funcionar com quaisquer dispositivos, sem se preocupar com as diferenças entre eles
 - o SO é responsável por tratar as diferenças
- **Nomeação uniforme:** todos os dispositivos devem ser nomeados da mesma maneira, sem distinções
 - exemplo: montagem de sistemas de arquivos no Unix
- **Tratamento de erros:** muitos erros de E/S podem ser tratados em baixo nível, sem chegar até o usuário
 - erros transientes
 - ideal é tratar o mais próximo possível do hardware

Técnicas para realizar E/S

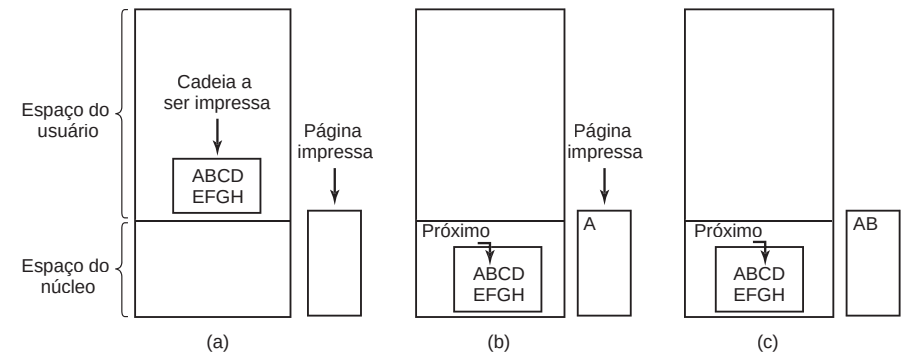
- E/S programada
- E/S orientada a interrupções
- E/S usando DMA

E/S programada (1)

- A CPU faz todo o trabalho
 - forma mais simples
- Exemplo: impressão de uma string
 - SO copia string para um buffer
 - dados do buffer são enviados, caractere a caractere, para a impressora (usando E/S mapeada em memória, p.ex.)
- Espera ocupada ou *polling*
- CPU fica muito tempo ociosa
 - solução adequada para sistemas embarcados ou se E/S é muito rápida (e ociosidade é pequena)

© 2023 Rafael Obelheiro (DCC/UDESC) Gerência de E/S SOP 25/87

E/S programada (2)



```
copy_from_user(buffer, p, cont);  
for (i=0; i < count; i++) {  
    while (*printer_status_reg != READY) ;  
    *printer_data_register = p[i];  
}  
return_to_user();
```

```
/* p é o buffer do núcleo */  
/* executa o laço para cada caractere */  
/* executa o laço até PRONTO */  
/* envia um caractere para a saída */
```

© 2023 Rafael Obelheiro (DCC/UDESC) Gerência de E/S SOP 26/87

E/S orientada a interrupções (1)

- Em vez da CPU esperar pelo dispositivo, ela inicia a operação de E/S e espera por uma interrupção para continuar
 - enquanto isso, outros processos podem ser executados

```
copy_from_user(buffer, p, count);  
enable_interrupts();  
while (*printer_status_reg != READY) ;  
*printer_data_register = p[0];  
scheduler();
```

(a)

(a) início da operação

```
if (count == 0) {  
    unblock_user();  
} else {  
    *printer_data_register = p[i];  
    count = count - 1;  
    i = i + 1;  
}  
acknowledge_interrupt();  
return_from_interrupt();
```

(b)

(b) tratamento da interrupção

© 2023 Rafael Obelheiro (DCC/UDESC) Gerência de E/S SOP 27/87

E/S orientada a interrupções (2)

- Mais eficiente que E/S programada
- CPU ainda é interrompida para cada caractere impresso

© 2023 Rafael Obelheiro (DCC/UDESC) Gerência de E/S SOP 28/87

E/S usando DMA

- CPU programa o controlador de DMA e é interrompida somente no final da operação
 - por exemplo, depois de todo o buffer impresso
- Em geral é mais eficiente
 - CPU é menos interrompida e fica livre para outros processos
 - controlador de DMA pode ser mais lento que a CPU

```
copy_from_user(buffer, p, count);  
set_up_DMA_controller();  
scheduler();
```

(a)

```
acknowledge_interrupt();  
unblock_user();  
return_from_interrupt();
```

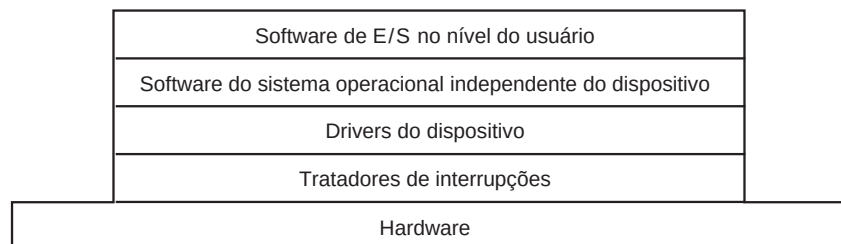
(b)

Sumário

- 1 Introdução
- 2 Princípios de hardware de E/S
- 3 Princípios do software de E/S
- 4 Camadas do software de E/S
- 5 Discos
- 6 Discos de estado sólido (SSDs)
- 7 E/S no Linux

Camadas do software de E/S

- Software de E/S é organizado em quatro camadas
 - cada camada tem uma função bem definida



Tratadores de interrupções

- As interrupções devem ser escondidas o máximo possível
 - uma forma de fazer isso é bloqueando o driver que iniciou uma operação de E/S até que uma interrupção notifique que a E/S foi completada
- Rotina de tratamento de interrupção cumpre sua tarefa
 - e então desbloqueia o driver que a chamou
- Tratamento de interrupções em software envolve
 - trocar de contexto (processo corrente → tratador)
 - tratar a interrupção
 - escolher um novo processo para executar (escalonador)
 - trocar de contexto (tratador → novo processo)

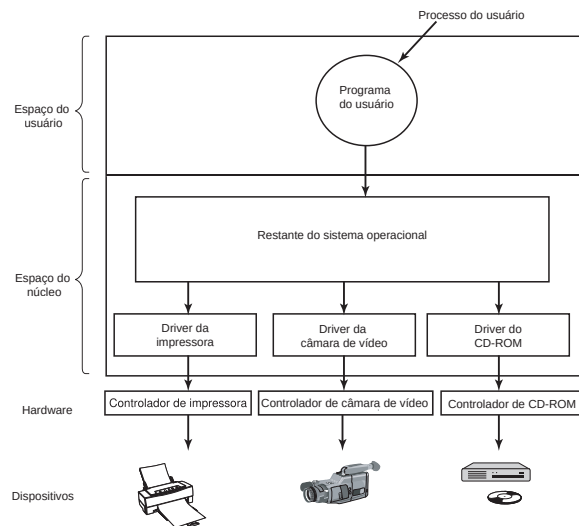
Drivers de dispositivos (1)

- Cada controlador possui registradores para comandos e status
- O número de registradores varia de dispositivo para dispositivo
- Cada dispositivo de E/S precisa de um código específico de tratamento (**driver do dispositivo**)
- Em geral, os drivers são escritos pelo fabricante
 - questões de confiabilidade/segurança
- É desejável que um driver trate de uma classe de dispositivos

Drivers de dispositivos (2)

- Duas grandes categorias:
 - dispositivos de bloco: disco, fita, ...
 - dispositivos de caractere: mouse, teclado, impressora, ...
- Em geral, os sistemas operacionais definem uma interface padrão para cada categoria
- Algumas funções de um driver:
 - tratar requisições abstratas de leitura ou gravação independente do dispositivo
 - inicializar o dispositivo
 - tratar necessidade de energia
 - tratar eventos

Drivers de dispositivos (3)



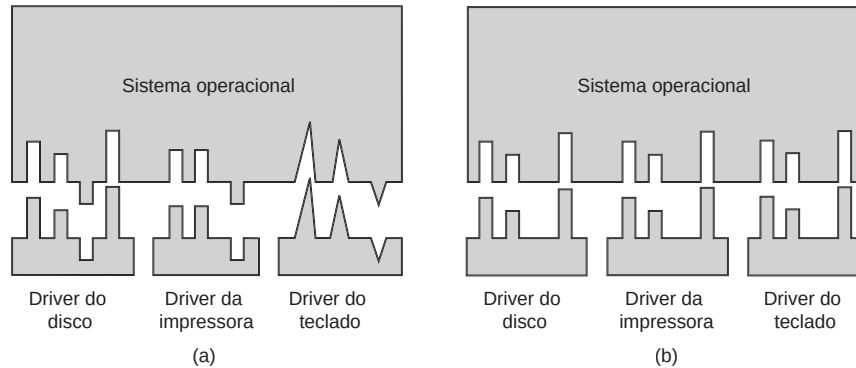
comunicação entre drivers e controladores é feita pelo barramento

E/S independente de dispositivo

Funções do software de E/S independente de dispositivo:

1. Interface uniforme para os drivers de dispositivos
2. Uso de buffers de E/S
3. Relatório de erros
4. Alocação e liberação de dispositivos dedicados
5. Tamanho de bloco independente de dispositivo

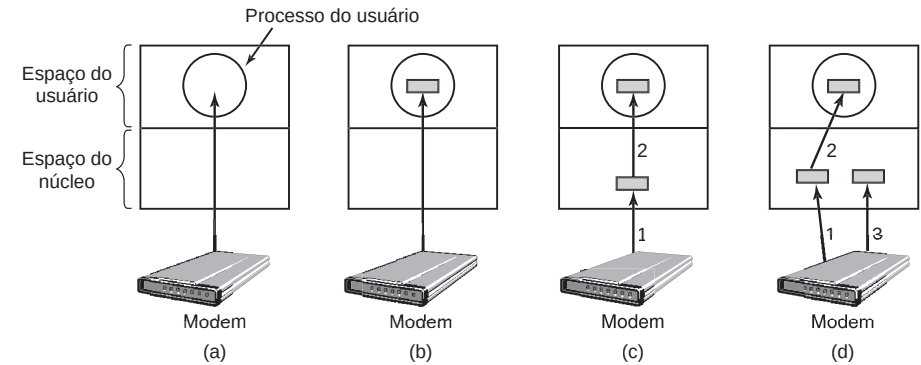
Interface uniforme para os drivers de dispositivos



(a) Sem uma interface padrão do driver

(b) Com uma interface padrão do driver

Uso de buffers de E/S



(a) Entrada sem uso de buffer

(b) Uso de buffer no espaço do usuário

(c) Uso de buffer no núcleo seguido de cópia para o espaço do usuário

(d) Uso de buffer duplo no núcleo

Relatórios de erros

- Erros de programação
 - ▶ escrita em um dispositivo de entrada (teclado)
 - ▶ fornecimento de endereços inválidos
 - ▶ atitude a ser tomada: retornar o código (tipo) de erro ocorrido ao processo envolvido
- Erros reais de E/S
 - ▶ tentativa de escrita em um bloco danificado
 - ▶ podem ser irrecuperáveis
 - ★ leitura do MBR do disco de boot, por exemplo
 - ▶ atitude deve ser decidida pelo driver

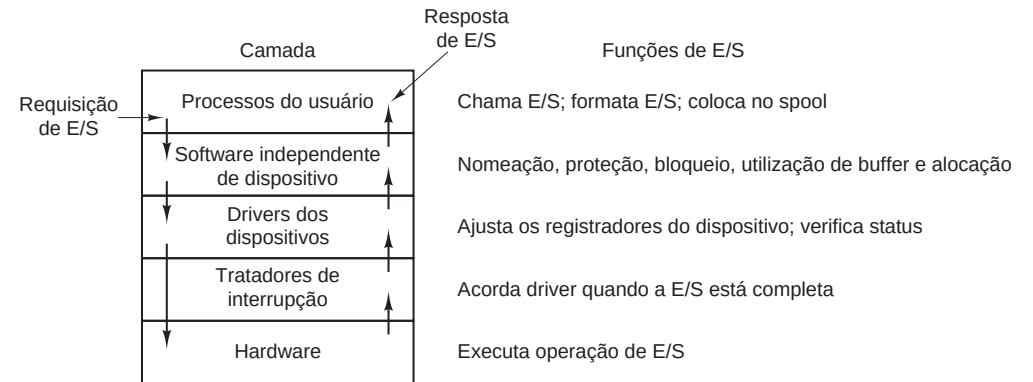
Alocação/liberação de dispositivos

- Alguns dispositivos, tais como gravadores de CD/DVD, podem ser usados por apenas um único processo por vez
- Associar chamadas de sistema diretamente ao dispositivo (`open`)
 - ▶ caso o dispositivo não possa ser alocado, a chamada pode falhar ou o chamador ser bloqueado
 - ★ decisão de projeto

Software de E/S no espaço de usuário

- Chamadas de sistema e funções de biblioteca que permitem ao usuário fazer E/S
 - `read`, `write`, `printf`, `scanf`, ...
- Spooling

Camadas do sistema de E/S: resumo



Sumário

- 1 Introdução
- 2 Princípios de hardware de E/S
- 3 Princípios do software de E/S
- 4 Camadas do software de E/S
- 5 Discos
- 6 Discos de estado sólido (SSDs)
- 7 E/S no Linux

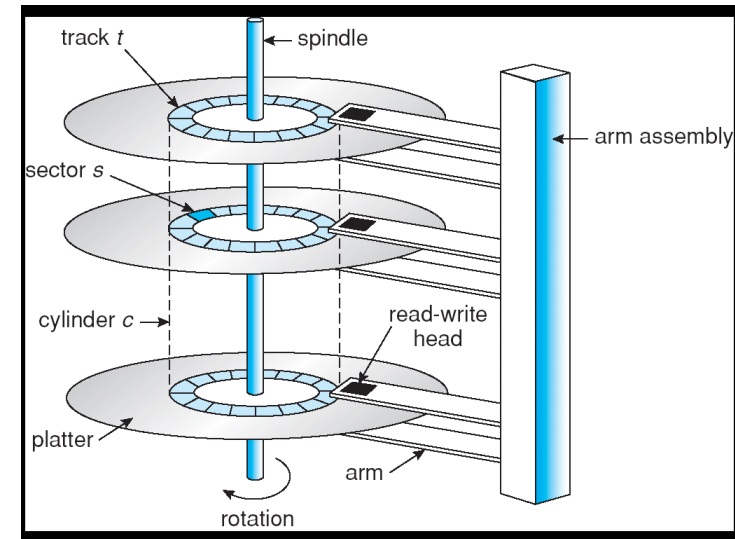
Discos magnéticos

- O disco é um dos dispositivos de E/S mais importantes
- Serve não apenas como memória secundária (de massa) como também oferece suporte a outras funções do SO
 - swapping
 - memória virtual
- Consideraremos aqui discos magnéticos (*hard disks*)
 - princípios de funcionamento similares aos de outros dispositivos como CD e DVD

Organização dos discos (1)

- Um disco é formado por um conjunto de pratos
 - superfícies cobertas por material ferromagnético
- A superfície do disco é magnetizada para armazenar os bits desejados
 - cabeça (cabeçote) de leitura e escrita converte bits de/para padrões magnéticos registrados na superfície do disco
 - há uma ou duas cabeças por superfície
- Cada disco é organizado em **trilhas** concêntricas
- Cada trilha é dividida em **setores** de tamanho fixo
 - trilhas e setores são definidos por formatação física pelo fabricante
- Um **cilindro** representa uma trilha em todas as superfícies

Organização dos discos (2)



Formato dos setores

- Cada setor possui o seguinte formato
 - preâmbulo: marca o início do setor
 - dados: geralmente em blocos de 512 bytes
 - ECC: um código de correção de erros (Reed-Solomon)
 - ★ permite corrigir erros de leitura em alguns bits de dados

Preâmbulo	Dados	ECC
-----------	-------	-----

Exemplos de parâmetros de disco

Parameter	IBM 360-KB floppy disk	WD 3000 HLFS hard disk
Number of cylinders	40	36,481
Tracks per cylinder	2	255
Sectors per track	9	63 (avg)
Sectors per disk	720	586,072,368
Bytes per sector	512	512
Disk capacity	360 KB	300 GB
Seek time (adjacent cylinders)	6 msec	0.7 msec
Seek time (average case)	77 msec	4.2 msec
Rotation time	200 msec	6 msec
Time to transfer 1 sector	22 msec	1.4 μ sec

- tempo médio de posicionamento melhorou 18 vezes
- taxa de transferência aumentou 16.000 vezes
- capacidade aumentou 800.000 vezes

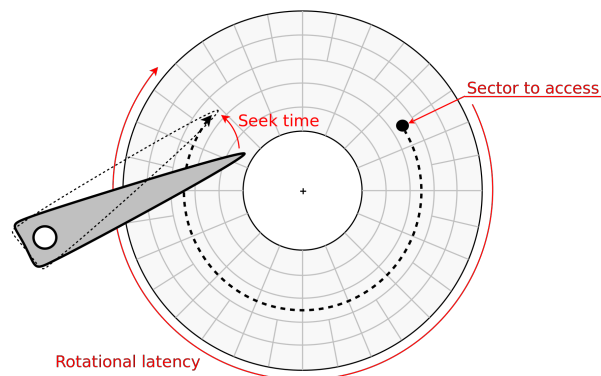
Acesso a dados

- Para acessar dados, é preciso localizar o bloco desejado
 - superfície, trilha, setor
- Existem dois métodos básicos de endereçamento
 - CHS (*Cylinder, Head, Sector*): usando as componentes
 - LBA (*Linear Block Addressing*): usando um no. de bloco
- O método mais usado atualmente é LBA
 - controlador é responsável por converter um número de bloco lógico nos parâmetros físicos adequados
 - permite mascarar detalhes como
 - ★ variações no número de setores por trilha: zonas
 - ★ setores defeituosos

Desempenho do disco

- Para ler ou escrever dados é necessário posicionar corretamente a cabeça de leitura e gravação
- O tempo da operação é dado por três componentes
 - **tempo de posicionamento (*seek*)**: tempo necessário para levar o braço até a trilha desejada
 - **latência rotacional**: tempo necessário para que o setor desejado passe sob o cabeçote
 - **tempo de transferência**: tempo necessário para transferir efetivamente os dados

Desempenho do disco



Fonte: <http://gudok.xyz/sspar/>

- Tempo de posicionamento domina

$$t_{\text{acesso}} = t_{\text{seek}} + t_{\text{lat}} + t_{\text{transf}}$$

Como determinar os componentes? (1)

- O tempo de seek é uma característica do disco, e tipicamente é fornecido pelo fabricante
 - muitas vezes se trabalha com um valor médio
- A latência rotacional depende da velocidade de rotação e de qual setor está passando embaixo do cabeçote quando o braço chega à trilha desejada
 - no melhor caso o setor desejado é o próximo
 - no pior caso, o setor desejado acabou de passar
 - na média, a latência é dada pelo tempo de meia rotação

Como determinar os componentes? (2)

- O tempo de transferência depende da velocidade de rotação e do tamanho e densidade dos setores

$$t_{transf} = \frac{b}{r \cdot N}$$

- ▶ b é a quantidade de bytes em um setor
- ▶ r é a velocidade de rotação (rps)
- ▶ N é a quantidade de bytes em uma trilha

Tempo médio de acesso

- O tempo médio de acesso é dado então por

$$t_{acesso} = t_{seek} + \frac{1}{2r} + \frac{b}{r \cdot N}$$

Cálculo do tempo médio (1)

- Qual o tempo médio necessário para ler um setor de um disco com os seguintes parâmetros?
 - ▶ tempo de seek médio: 10 ms
 - ▶ velocidade de rotação: 10.000 rpm
 - ▶ bytes/setor: 512
 - ▶ bytes/trilha: 32.768

$$t_{acesso} = 0,01 + \frac{1}{2 \times \frac{10000}{60}} + \frac{512}{\frac{10000}{60} \times 32768} = 13,09 \text{ ms}$$

Cálculo do tempo médio (2)

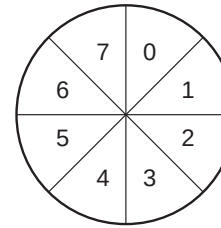
- Qual o tempo médio necessário para ler cinco setores consecutivos do disco usado como exemplo? (slide 48)
- Parâmetros:
 - ▶ tempo de seek médio: 4,2 ms
 - ▶ tempo de rotação: 6 ms
 - ▶ bytes/setor: 512
 - ▶ setores/trilha: 63

$$t_{acesso} = 4,2 + \frac{6}{2} + 6 \times \frac{5 \times 512}{63 \times 512} = 7,68 \text{ ms}$$

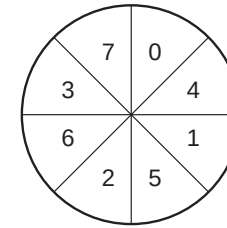
Entrelaçamento (1)

- Alguns discos não conseguem ler setores adjacentes de forma consecutiva
 - ▶ devido à transferência do bloco de dados do buffer do controlador para a memória, o próximo setor passa sob o cabeçote antes que a leitura possa iniciar
 - ▶ isso implica uma rotação adicional para cada setor
- Uma solução é introduzir setores entre setores logicamente adjacentes
 - ▶ dá tempo para que o próximo setor seja lido

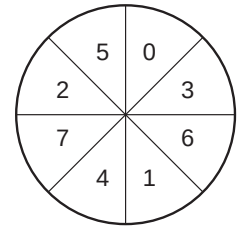
Entrelaçamento (2)



(a)



(b)



(c)

- (a) sem entrelaçamento
- (b) entrelaçamento simples
- (c) entrelaçamento duplo

Escalonamento de disco

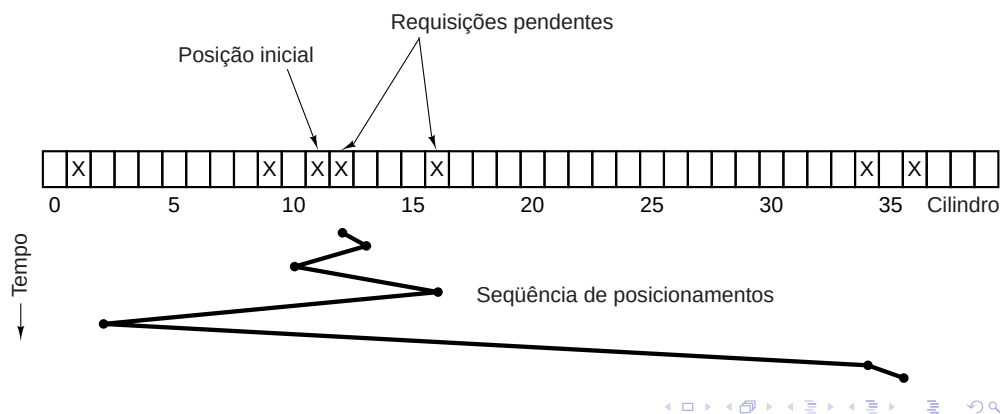
- Como o tempo de posicionamento domina o tempo de acesso, reduções nesse tempo melhoram o desempenho dos acessos a disco
- Embora o tempo de posicionamento seja uma característica do disco, é possível mudar a ordem em que as requisições de leitura e escrita são atendidas para minimizar os deslocamentos do braço
- Essa é a função dos **algoritmos de escalonamento de disco**
 - ▶ FCFS
 - ▶ SSF
 - ▶ elevador

FCFS (*First Come, First Served*)

- As requisições são atendidas na ordem de chegada
- Consideremos um exemplo
 - ▶ disco tem 40 cilindros
 - ▶ está sendo atendida uma requisição no cilindro 11
 - ▶ chegam requisições para os cilindros 1, 36, 16, 34, 9 e 12 (nessa ordem)
 - ▶ a distância total percorrida para atender o conjunto de requisições é de 111 cilindros

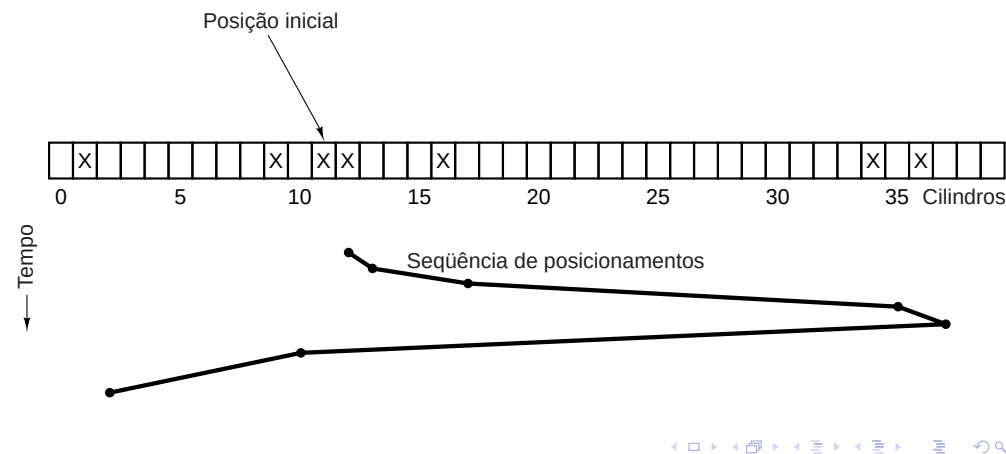
SSF (Shortest Seek First)

- A requisição seguinte a ser atendida é aquela que estiver mais próxima do cilindro atual
- Algumas requisições podem levar muito tempo para serem atendidas
 - concentração de requisições em uma região do disco
- Exemplo



Algoritmo do elevador

- Atende a todas as requisições em uma direção
- Quando chega ao final, muda a direção e atende às demais requisições
- Exemplo (inicialmente subindo)



Diferença entre os algoritmos

FCFS		SSF		elevador (↑)	
cilindro	distância	cilindro	distância	cilindro	distância
11	—	11	—	11	—
1	10	12	1	12	1
36	35	9	3	16	4
16	20	16	7	34	18
34	18	1	15	36	2
9	25	34	33	9	27
12	3	36	2	1	8
total	111 cil	total	61 cil	total	60 cil
média	18,5 cil/req	média	10,2 cil/req	média	10 cil/req

- SSF e elevador reduzem em quase 50% a distância total percorrida em relação ao FCFS
 - para esse conjunto de requisições

Sumário

- 1 Introdução
- 2 Princípios de hardware de E/S
- 3 Princípios do software de E/S
- 4 Camadas do software de E/S
- 5 Discos
- 6 Discos de estado sólido (SSDs)
- 7 E/S no Linux

SSDs

- Discos de estado sólido (*solid state disks*, SSDs) vêm tendo adoção crescente nos últimos anos
 - substituindo ou complementando HDs (como cache)
- Usam chips de memória flash em vez de pratos rotativos
 - interface é quase a mesma de discos rígidos
- Vantagens
 - sem partes móveis → menor ruído, menor consumo de energia, menor risco de danos mecânicos
 - melhor desempenho, em geral
 - ★ sem necessidade de deslocamento de braço nem latência rotacional
- Desvantagens
 - capacidade geralmente menor
 - custo por GB maior
 - sujeito a desgaste → menor durabilidade (nº limitado de escritas), perda de desempenho com o tempo

Exemplo de operação

- Supondo um bloco com 4 páginas

	iiii	inicial: páginas inválidas (i)
apaga	→ AAAA	páginas 0–3 apagadas (A)
prog(0)	→ VAAA	programa pág 0, fica válida (v)
prog(0)	→ erro	não pode reprogramar pág 0
prog(1)	→ VVAA	programa pág 1
apaga	→ AAAA	páginas 0–3 apagadas (A)
- Para reescrever uma página no mesmo lugar, seria necessário apagar o bloco, ler e reprogramar também todas as páginas válidas do bloco
 - perda de desempenho
 - aumento do desgaste do bloco

Organização da memória flash

- Um chip de memória flash é dividido em **blocos**, que são subdivididos em **páginas**
 - blocos são grandes (tipicamente 128 KB a 2 MB)
 - páginas são pequenas (tipicamente 1 KB a 8 KB)
- Existem 3 operações básicas: leitura (de página), apagamento (de bloco) e programação (de página)
 - para que um bloco seja usado, ele deve primeiro ser apagado
 - depois que o bloco foi apagado, cada uma de suas páginas só pode ser programada uma vez
 - ★ não é possível reescrever uma página
 - latências: leitura: 25–75 μ s, programação: 200–1350 μ s, apagamento: 1,5–4,5 ms

Flash translation layer (FTL)

- Um SSD oferece (quase) a mesma interface de um disco rígido → leitura e escrita de setores
 - em geral, uma página contém vários setores
 - ★ p.ex., uma página de 4 KB contém 8 setores de 512 B
- Para minimizar a reprogramação de páginas, o controlador do SSD implementa um mapeamento de setores/páginas lógicas em páginas físicas
 - *flash translation layer* (FTL)
- Quando é solicitada uma reescrita de página, os dados são gravados em uma página disponível, e o mapeamento atualizado
 - redução de desgaste (*wear leveling*)
- Leituras usam o mapeamento para encontrar a página correta

Coleta de lixo

- O que acontece se não houver páginas disponíveis para escrita?
- SSDs costumam ser superprovisionados, contendo uma reserva de blocos ($\approx 20\%$) para facilitar o processo
- Com o tempo, a tendência é que os blocos tenham páginas válidas e inválidas (que não estão mais em uso)
 - ▶ o mapeamento da FTL permite determinar as páginas em uso
 - ▶ SSDs também oferecem uma operação TRIM para sinalizar que um setor não está mais em uso
 - ★ quando um arquivo é removido, o SA deve usar TRIM para indicar os setores liberados → desnecessário em HDs
- É possível copiar as páginas válidas de blocos parcialmente em uso para um novo bloco e apagar os antigos, que passam a ficar integralmente disponíveis → **coleta de lixo**

SATA/AHCI vs PCIe/NVMe

- As interfaces mais comuns para SSDs são SATA (6 Gb/s) e PCIe (32 Gb/s)
 - os respectivos protocolos são AHCI (SATA) e NVMe (PCIe)
- Dispositivos PCIe/NVMe são potencialmente bem mais rápidos
 - exigem mais do SO para conseguir atingir o desempenho oferecido
- SATA oferece uma única fila de comandos, com até 32 requisições
 - com múltiplos núcleos, a fila vira um gargalo de sincronização
- NVMe oferece múltiplas (até 64K) filas de comandos, cada uma com até 64K requisições de E/S
 - ideia é usar uma fila por núcleo, eliminando gargalo
 - múltiplas filas, com mais requisições, permitem explorar melhor o paralelismo interno do SSD

HDs vs SSDs: comparação quantitativa

Dispositivo	aleatório		sequencial	
	leitura (MB/s)	escrita (MB/s)	leitura (MB/s)	escrita (MB/s)
Samsung 840 Pro SSD	103	287	421	384
Seagate 600 SSD	84	252	424	374
Intel SSD 335 SSD	39	222	344	354
Seagate Savvio 15.3 HDD	2	2	223	223

- SSDs são muito mais rápidos que HDs para acesso aleatório
- A diferença entre SSDs e HDs é menor para acesso sequencial
- Nos SSDs, no acesso aleatório o desempenho de escrita é superior ao de leitura porque FTL transforma acesso aleatório em sequencial
- A diferença entre acesso aleatório e sequencial mostra que em SSDs também há benefício na minimização de acessos aleatórios

HDs vs SSDs: algumas considerações

- SSDs vêm tomando o lugar de HDs como dispositivos de armazenamento de propósito geral
- HDs ainda são a melhor opção para armazenamento de alta capacidade, com acesso não muito frequente ou sequencial
- HDs também podem ser mais indicados para escrita em streaming, devido ao desgaste dos SSDs

Sumário

- 1 Introdução
- 2 Princípios de hardware de E/S
- 3 Princípios do software de E/S
- 4 Camadas do software de E/S
- 5 Discos
- 6 Discos de estado sólido (SSDs)
- 7 E/S no Linux

Acesso a dispositivos de E/S

- No Unix/Linux, cada dispositivo de E/S pode ser acessado por um **arquivo especial**
 - ▶ geralmente em `/dev`
 - ★ `/dev/sda`: primeiro disco SCSI/SATA
 - ★ `/dev/psaux`: porta de mouse PS/2
 - ▶ uma aplicação pode usar as mesmas chamadas de sistema válidas para arquivos
 - ★ `open()`, `read()`, `write()`, `close()`, ...
- Existem duas grandes categorias de arquivos especiais
 - ▶ **de bloco**: disco, fita
 - ▶ **de caracter**: teclado, mouse, impressora, rede
- Cada arquivo especial possui dois **números de dispositivo**
 - ▶ **principal** (*major*): identifica o driver
 - ▶ **secundário** (*minor*): identifica o dispositivo (dentre vários)

Exemplos de arquivos especiais

- Disco rígido SATA com duas partições
 - ▶ `b` indica dispositivo de bloco
 - ▶ número principal de dispositivo 8
 - ▶ os números secundários de dispositivo indicam o disco inteiro (`sda`, 0) ou cada partição (`sda1`, `sda2`)

```
brw-rw---- 1 root disk 8, 0 Ago 28 15:01 sda
brw-rw---- 1 root disk 8, 1 Ago 28 15:01 sda1
brw-rw---- 1 root disk 8, 2 Ago 28 15:01 sda2
```

- Pseudoterminais (três janelas abertas em um emulador de terminal)
 - ▶ `c` indica dispositivo de caracter
 - ▶ número principal de dispositivo 136
 - ▶ os números secundários de dispositivo indicam cada janela

```
crw--w---- 1 rro tty 136, 0 Ago 28 15:01 /dev/pts/0
crw----- 1 rro tty 136, 1 Ago 28 15:01 /dev/pts/1
crw----- 1 rro tty 136, 2 Ago 28 15:01 /dev/pts/2
```

Escalonamento de disco no Linux

- Funções do escalonador de disco
 - ▶ **fundir requisições**: quando uma requisição é recebida, o escalonador verifica se existe uma requisição adjacente; se houver, as duas requisições são combinadas em uma
 - ★ *request coalescing*
 - ★ minimiza seeks
 - ▶ **ordenar requisições**: ordena as requisições com o propósito de minimizar o tempo de seek e evitar inanição
- Princípios gerais
 - ▶ cada dispositivo de bloco tem uma (ou várias) filas de requisições pendentes
 - ▶ cada dispositivo tem uma fila de despacho (*dispatch*), que contém as requisições enviadas ao dispositivo
 - ▶ um escalonador de disco manipula fila(s) de requisições pendentes para alimentar a fila de despacho
 - ★ em algoritmos simples, a fila de requisições pendentes e a fila de despacho são uma só

Escalonadores de disco no Linux

- Elevador do Linus
- Escalonador com deadlines
- Escalonador antecipatório
- Escalonador com enfileiramento completamente justo
- Escalonador noop
- O escalonador em uso no dispositivo DEV pode ser examinado ou modificado via `/sys/block/DEV/queue/scheduler`

Exemplo

```
$ cat /sys/block/sda/queue/scheduler
noop deadline [cfq]
# echo deadline >/sys/block/sda/queue/scheduler
```

O elevador do Linus

- Usado no kernel 2.4
- Realiza ordenação e fusão de requisições, da seguinte forma:
 1. Se uma requisição para um setor adjacente já se encontra na fila, a requisição existente e a nova são fundidas
 - ★ fusão pode ser recursiva, até um limite de blocos por requisição
 2. Se uma requisição na fila for suficientemente antiga, a nova requisição entra no final da fila
 - ★ tenta evitar inanição de requisições, mas não funciona muito bem
 3. Caso contrário, insere a requisição na fila seguindo o algoritmo do elevador

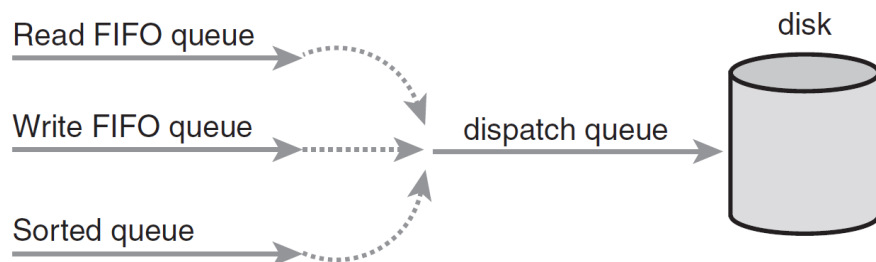
Inanição de requisições

- O elevador do Linus está sujeito a inanição
- Concentração de requisições em uma região do disco
- Escritas causando inanição de leituras
 - ▶ operações de escrita são geralmente assíncronas
 - ★ dados são copiados para um buffer no kernel antes de serem gravados no disco
 - ★ a aplicação não fica esperando o resultado
 - ★ leituras podem usar o buffer
 - ▶ operações de leitura são geralmente síncronas
 - ★ aplicação fica bloqueada esperando os dados do disco
 - ▶ requisições de leitura tendem a ser encadeadas
 - ★ uma requisição precisa esperar a anterior
 - ▶ latência de leitura afeta mais perceptivelmente o desempenho do que latência de escrita

O escalonador com deadlines

- Introduzido no kernel 2.6
- Objetivo é reduzir os problemas de inanição do elevador do Linus
- Cada requisição possui um tempo de expiração
 - ▶ default: 500 ms para leituras, 5 s para escritas
- Mantém três filas de requisições
 - ▶ fila ordenada (como no elevador)
 - ▶ fila FIFO de leituras
 - ▶ fila FIFO de escritas
- Uma requisição é inserida na fila ordenada e no final da fila FIFO correspondente
- Geralmente, requisições são passadas da fila ordenada para a fila de despacho
 - ▶ se a requisição no início de uma das filas FIFO tiver expirado, ela é colocada na fila de despacho

O escalonador com deadlines



O escalonador antecipatório

- O escalonador com deadlines pode incorrer em seeks longos devido à expiração de requisições
 - se o sistema está num ciclo de escritas em uma região do disco e uma requisição de leitura expira, é necessário levar o braço até a trilha da leitura e depois retornar para as escritas
 - se as requisições de leitura forem encadeadas, isso pode ocorrer diversas vezes
 - vazão global do sistema é prejudicada
- O escalonador antecipatório adiciona uma heurística para minimizar o problema
 - quando uma requisição de leitura é atendida, o escalonador espera um tempo (default: 6 ms) antes de retornar à ordem normal de atendimento
 - se outra requisição de leitura para uma região adjacente chegar durante o tempo de espera, ela é atendida imediatamente
- Requer estatísticas por processo para antecipação funcionar bem

Escalonador com enfileiramento completamente justo

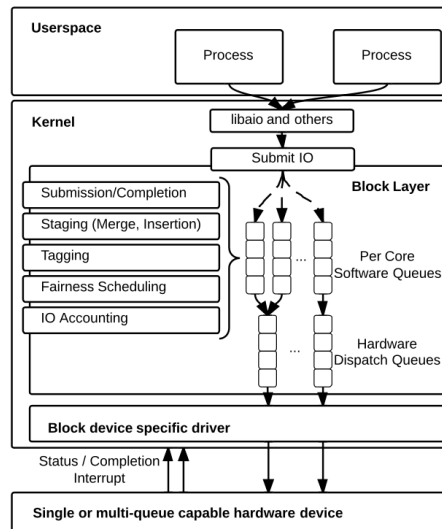
- *Completely fair queueing (CFQ)*
- Cada processo tem uma fila de requisições pendentes
 - uma nova requisição é ordenada e fundida com as requisições pendentes do mesmo processo
- O escalonador atende as filas em round-robin
 - um número de requisições de um processo (default: 4) são atendidas, e depois o escalonador escolhe a fila de outro processo
- Cada processo obtém uma fatia justa do disco
- Escalonador default no kernel 2.6

Escalonador noop

- Noop → *No operation*
- Realiza apenas fusão de requisições adjacentes
- Usado em dispositivos de acesso realmente direto ($t_{seek} \rightarrow 0$)
 - pendrives e outros “discos” de estado sólido

Múltiplas filas na camada de bloco

- Uma fila de submissão por núcleo, filas de despacho conforme a capacidade do hardware (idealmente uma por núcleo)



Fonte: <https://kernel.dk/systor13-final18.pdf>

Escalonadores que aproveitam múltiplas filas

- mq-deadline
 - escalonador com deadlines adaptado para múltiplas filas
- Budget Fair Queueing (BFQ)
 - mantém filas de requisições por processo
 - usa heurísticas para calcular um “orçamento” de setores que um processo pode transferir
- Kyber
 - mantém filas de operações síncronas (leituras) e assíncronas (escritas)
 - o número de requisições nas filas de despacho é limitado para atingir latências alvo
 - ★ default: 2 ms para leitura e 10 ms para escrita
 - o tamanho das filas é ajustado dinamicamente de acordo com as latências observadas
- BFQ é melhor com dispositivos mais lentos, Kyber com dispositivos mais rápidos
 - com SSDs, é comum não usar nenhum escalonador (none)

Bibliografia Básica

- Andrew S. Tanenbaum e Herbert Bos.**
Sistemas Operacionais Modernos, 4ª Edição. Capítulos 5 e 10.
Pearson Prentice Hall, 2016.
- Carlos A. Maziero.**
Sistemas Operacionais: Conceitos e Mecanismos. Capítulos 19 a 21.
Editora da UFPR, 2019.
<http://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start>
- Robert Love.**
Linux Kernel Development, 3rd Edition. Capítulo 14.
Pearson, 2010. (Cópia local disponível no Moodle)
- Remzi Arpaci-Dusseau e Andrea Arpaci-Dusseau.**
Operating Systems: Three Easy Pieces. Capítulo 44 (Flash-Based SSDs).
<https://www.ostep.org>